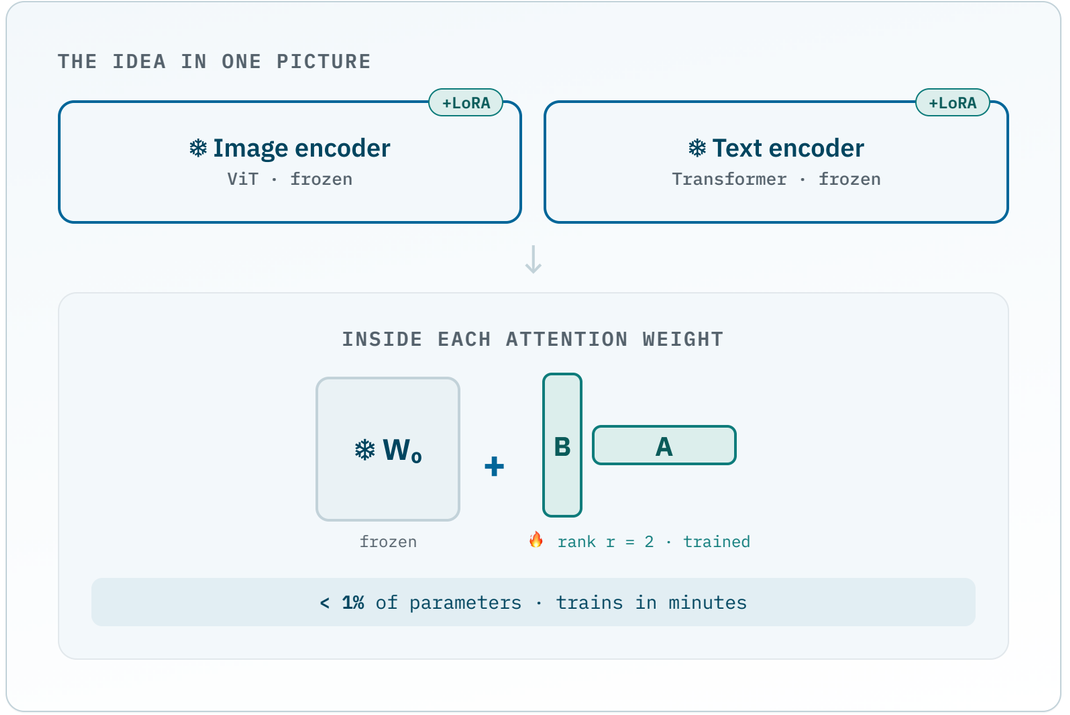
DL4NLP REPRODUCIBILITY & EXTENSION

Low-Rank Few-Shot Adaptation of Vision–Language Models

Understanding, reproducing, and extending **CLIP-LoRA** — adapting CLIP from just a handful of labelled images.

CLIP · Radford 2021 | CLIP-LoRA · Zanella 2024 | 11 datasets · few-shot



Hasan Berke Bankoglu · Zarek Asif

00 AGENDA

What we'll walk through

01 **What is CLIP?**

The vision–language model we adapt — how it's trained and how zero-shot classification works.

02 **What is CLIP-LoRA?**

Low-rank adapters inside CLIP's attention blocks — the design choices and why they work.

03 **Our reproduction**

Setup, scope, results so far **(in progress)**, and the challenges we hit.

04 **Extension & outlook**

A regularization term that anchors the adapted model to zero-shot CLIP — motivation & plan.

00 THE PROBLEM

Adapting a giant model with only a few examples

Vision–language models like **CLIP** are pre-trained once on hundreds of millions of image–text pairs. To use one on **your** task you only have a handful of labelled images — the **few-shot** regime.

The question this paper answers: **what is the cheapest, most effective way to specialise CLIP from 1–16 examples per class** — without fine-tuning the whole network or running slow prompt-optimization?

FULL FINE-TUNING

Too many parameters

Millions of weights, easy to overfit on a few shots, heavy to store per task.

PROMPT TUNING

Slow to train

Methods like PLOT++ need many hours of optimization per task.

CLIP-LORA

Few params · fast · accurate

Train tiny low-rank matrices inside attention — minutes, not hours.

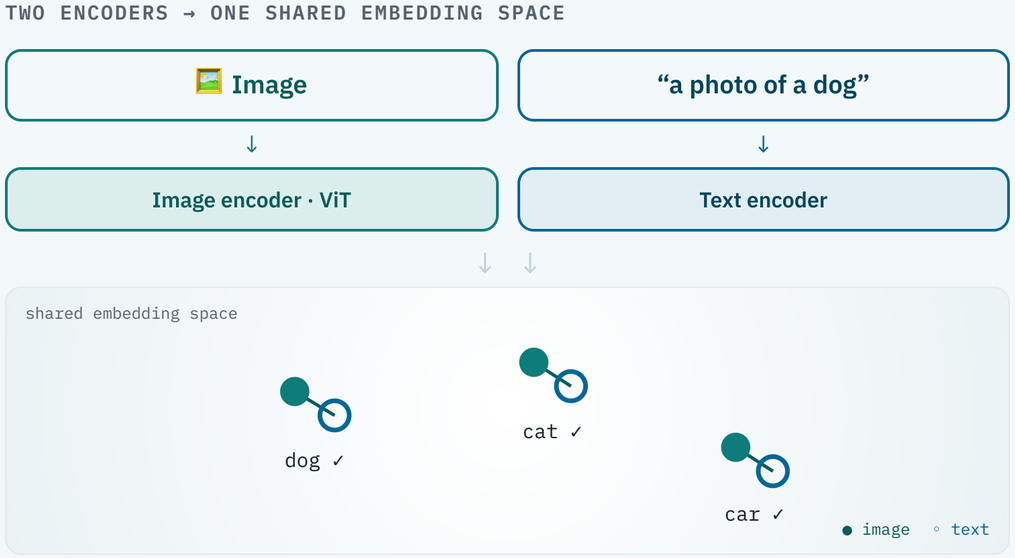
01 PART ONE

What is CLIP?

The foundation model we're adapting — how it learns to connect images and language.

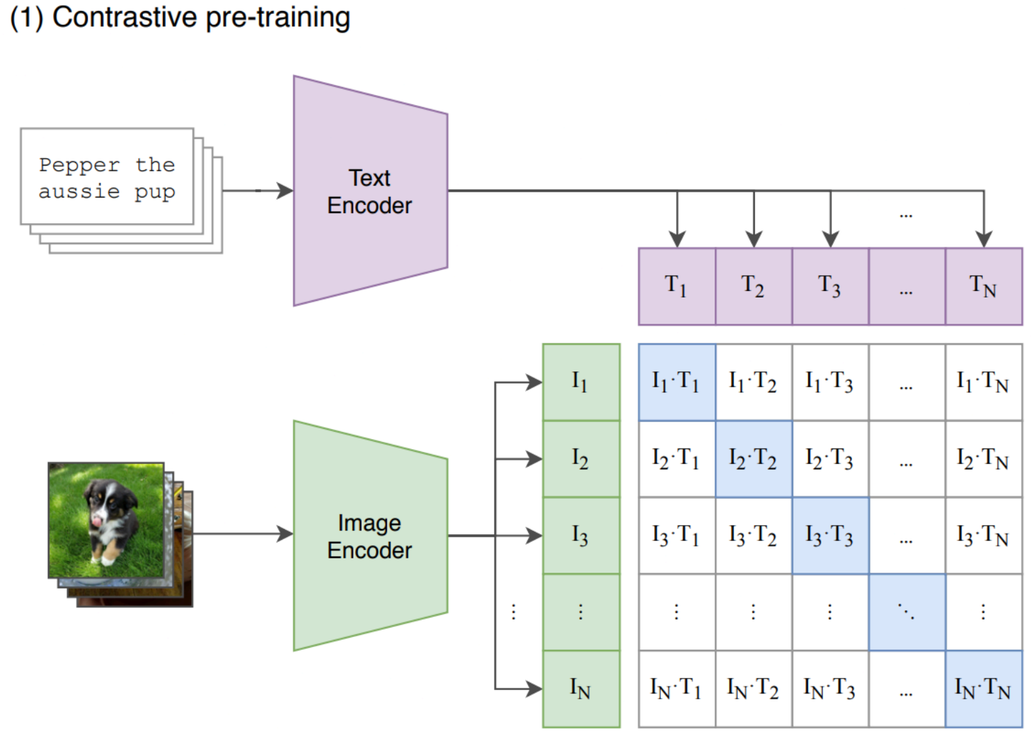
One model, two encoders, a shared space

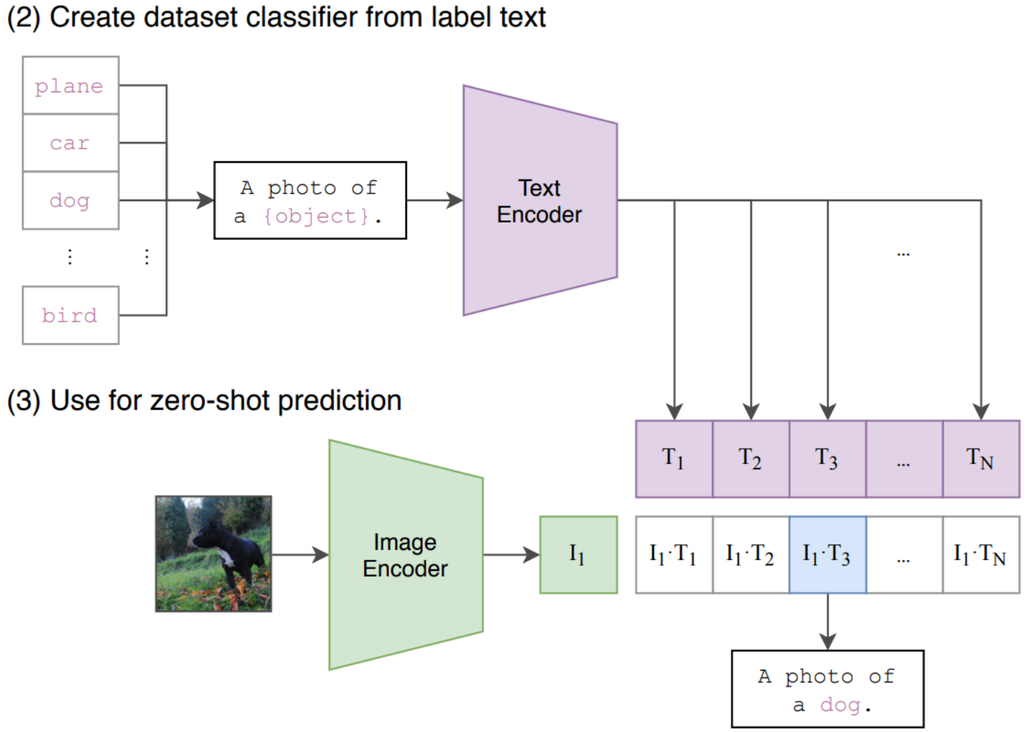
- **Two encoders.** An **image encoder** (a Vision Transformer) and a **text encoder** (a Transformer) map their inputs into one shared embedding space.
- **Trained on scale.** ~400M image–text pairs scraped from the web — no manual class labels needed.
- **Aligned by meaning.** An image and its caption land **close together**; unrelated pairs land far apart.
- **The payoff.** Comparing image and text embeddings lets CLIP classify images it was never explicitly trained on — **zero-shot**.



Matched image–text pairs land **close together** — that proximity **is** the prediction signal.

How CLIP works, end to end





1 **Contrastive pre-training.** Image & text encoders learn to match the correct pairs in each batch.

2 **Build a classifier from labels.** Class names go into a prompt → text embeddings.

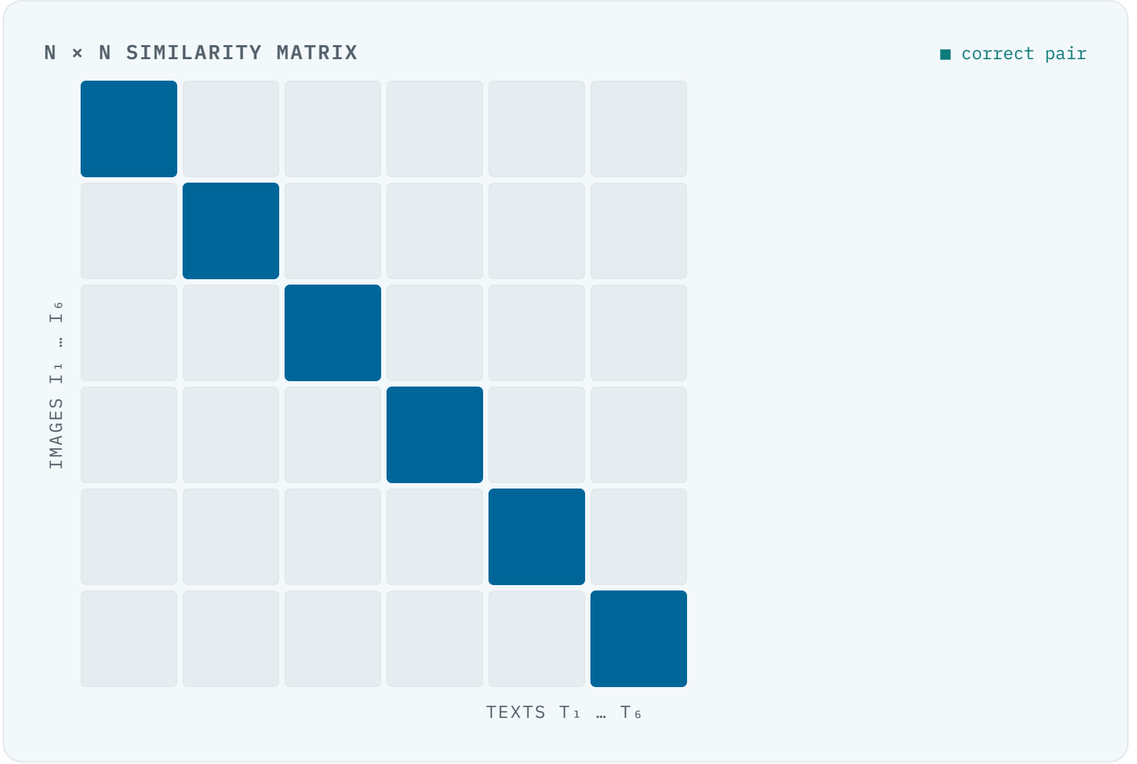
3 **Zero-shot prediction.** Pick the class whose text embedding is most similar to the image.

Match the right caption out of the whole batch

For a batch of **N** image–text pairs, CLIP computes all **N × N** cosine similarities. The training objective pulls each image toward **its own** caption and pushes it away from the other N−1.

$$\text{sim}(I, T) = (I \cdot T) \,/\, (\|I\| \, \|T\|)$$

A symmetric cross-entropy loss over rows **and** columns maximises the **diagonal** (correct pairs) of the similarity matrix.



01 CLIP · ZERO-SHOT CLASSIFICATION

Classify by comparing to text prompts

PROMPT TEMPLATE — NO TRAINING, JUST TEXT

“a photo of a cat ” “a photo of a dog ” “a photo of a forest ” …one per class

Strong out of the box — but on **fine-grained** tasks (aircraft types, satellite terrain) zero-shot alone isn't enough. That's where few-shot **adaptation** comes in.

02 PART TWO

What is CLIP-LoRA, and how does it work?

From low-rank adapters to a fast, fixed recipe for few-shot adaptation.

02 WHERE CLIP-LORA SITS

Two families of few-shot adaptation

FAMILY A · PROMPT LEARNING

Optimise the text side

Learn continuous prompt tokens fed to the text encoder — **CoOp, CoCoOp, PLOT++, KgCoOp, ProGrad, MaPLe**.

Cost: back-propagation through the text encoder ⇒ slow (hours).

FAMILY B · ADAPTERS

Add small trainable modules

Bolt lightweight layers onto frozen features — **CLIP-Adapter, Tip-Adapter-F, TaskRes**.

Limit: often touch only one modality; method-specific tuning.

CLIP-LoRA **A third path.** Inject **low-rank adapters directly into the attention weights** of **both** encoders. One fixed recipe, no per-task tuning, trains in minutes.

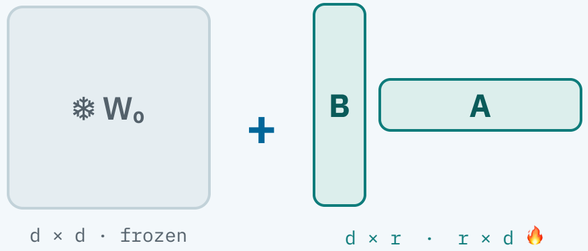
LoRA — freeze the big matrix, train a tiny one

Instead of updating a weight matrix **W**, LoRA **freezes it** and learns a **low-rank update** ΔW = **B A**, where B and A are skinny matrices of rank **r ≪ d**.

$$h = W_0 x + B A x$$

- **W_0 frozen** ❄ — pretrained knowledge is preserved.
- **Only B, A train** 🔥 — a few thousand parameters per layer.
- **Zero inference cost** — B A can be merged back into W after training.

LOW-RANK DECOMPOSITION



rank r = 2 in CLIP-LoRA ⇒ <1% of params trained

LoRA inside every attention block of both encoders

Inside each self-attention block, the query / key / value projections are the levers. CLIP-LoRA attaches a low-rank update to **Wq, Wk, Wv**:

`+ LoRA → Wq` `+ LoRA → Wk` `+ LoRA → Wv`

- **Both modalities adapt.** Vision *and* text encoders move together — key for fine-grained tasks.
- **Every level.** Adapters sit at all encoder depths (the “All” placement).
- **One recipe, all tasks.** The same hyper-parameters are used for every dataset — no per-task search.

One fixed configuration, kept across all 11 datasets

HYPER-PARAMETER	VALUE
Rank r	2
Target matrices	Wq, Wk, Wv
Encoders	vision + text
Placement	all layers
Dropout p	0.25
Learning rate	2e-4
Scheduler	cosine
Batch size	32
Iterations	500 × shots
Prompt	“a photo of a [class]”

The strength of the method is its **simplicity**: no per-dataset hyper-parameter search, no validation-set tricks. The same ten settings run everywhere.

r = 2
tiny rank

24 GB
fits one GPU

Results are averaged over **3 random seeds (1, 2, 3)** — the convention we follow exactly in our reproduction.

Faster than prompt learning, competitive accuracy

TRAINING TIME · 16-SHOT IMAGENET · SINGLE A100

METHOD	TRAIN TIME
PLOT++	15 h 30
ProGrad	3 h 20
CoOp	2 h 00
CLIP-LoRA	50 min

- **≈ 18× faster** than PLOT++ to train, on the same task.
- **State-of-the-art accuracy** across the 11-dataset few-shot benchmark, with no per-task tuning.
- **Consistent across backbones** — ViT-B/16, ViT-B/32, ViT-L/14.
- **Our reproduction** (16-shot ImageNet, single A40): ≈104 min B/16, 92 min B/32, 191 min L/14 — same method, slower GPU than the paper's A100. Whole project ≈ 275 GPU-h.

Takeaway: a generic, well-understood NLP/LLM technique — LoRA — transfers cleanly to vision–language adaptation and beats specialised methods on speed.

03 PART THREE

Reproducing the paper

Our setup, what we reproduced, the results — and the challenges along the way.

03 REPRODUCTION · SETUP

Built on the authors' code, our own runners

BASE

Official repo

Forked `MaxZanella/CLIP-LoRA` and reproduced the default configuration faithfully.

PROTOCOL

Seeds 1·2·3

Every experiment averaged over three seeds, exactly as the paper reports.

HARDWARE

Single GPU

Runs sized to fit a single GPU; currently running on an NVIDIA A40 (48 GB).

OUR REPRODUCIBILITY SCAFFOLDING

`run_sanity_check.sh` `run_table_3.sh` `run_table_4.sh` `generate_fig3_jobs.py` `run_jobs.sh · failure-tolerant` `aggregate_results.py`

Every run logs `dataset, backbone, shots, seed, rank, targets, encoder, placement, dropout, accuracy, status` into a single `clip_lora_results.csv`.

What we set out to reproduce

Main tables · ViT-B/16, B/32 & L/14

phase 1

CLIP-LoRA across **10 datasets** × shots {1,2,4,8,16} × 3 seeds.

`10 × 5 × 3 = 150 runs per backbone (3 backbones)`

Figure 3 ablations

phase 2

Rank × attention-matrix set × encoder choice, plus placement — on ImageNet, StanfordCars, EuroSAT at 4-shot.

`129 combos × 3 datasets = 387 runs (1 seed)`

Work split

parallel

Two of us run backbones in parallel; Fig.3 jobs split round-robin so rank/dataset load is balanced.

Scope note

Full grid for **10 of the paper's 11 datasets**; SUN397 dropped (dead URL / parquet-only mirror).

How we split the work

HB **Hasan Berke Bankoglu**

- **Table 3** — ViT-B/16 main results
- **Table 5** — ViT-L/14 main results
- **Figure 3** — ImageNet & EuroSAT ablations
- **Extension** — KL regularization to zero-shot CLIP

ZA **Zarek Asif**

- **Table 4** — ViT-B/32 main results
- **Figure 3** — StanfordCars ablations

Each backbone is presented on its own slide next — every reproduced average lands within **≈0.3%** of the paper across all shot counts and 10 datasets (SUN397 excluded).

RAN BY · H. B. BANKOGLU

Few-shot results · ViT-B/16

ours paper Δ = ours−paper

Shots	ImgNet	Airc	Euro	Cars	Food	Pets	Flwr	Calt	DTD	UCF	Avg
1	70.4 70.4	29.8 30.2	71.1 72.3	69.8 70.1	84.7 84.3	91.9 92.3	82.3 83.2	93.9 93.7	54.0 54.3	76.0 76.3	72.4 72.7 -0.3
2	70.8 70.8	32.6 33.2	82.0 82.7	73.4 73.2	83.3 83.2	90.7 91.3	89.8 89.8	94.8 94.6	59.9 59.9	80.1 80.0	75.7 75.9 -0.1
4	71.5 71.4	38.0 37.9	85.3 84.9	77.1 77.4	83.0 82.7	90.5 91.0	93.3 93.7	95.0 95.2	63.7 63.8	81.1 81.1	77.8 77.9 -0.1
8	72.4 72.3	45.4 45.7	89.1 89.7	82.1 82.1	83.1 83.1	91.3 91.7	96.1 96.3	95.7 95.6	67.5 67.5	84.0 84.1	80.7 80.8 -0.1
16	73.6 73.6	54.9 54.7	92.3 92.1	86.3 86.3	84.1 84.2	92.3 92.4	97.9 98.0	96.2 96.4	71.8 72.0	86.4 86.7	83.6 83.6 -0.0

Top-1 accuracy (%), 3-seed avg (seeds 1·2·3) · paper = Zanella & Ben Ayed (2024), Table 3 · averages over the 10 datasets we ran (SUN397 excluded)

RAN BY · Z. ASIF

Few-shot results · ViT-B/32

ours paper Δ = ours−paper

Shots	ImgNet	Airc	Euro	Cars	Food	Pets	Flwr	Calt	DTD	UCF	Avg
1	65.3 65.2	22.9 22.9	67.9 67.1	63.6 63.5	78.5 78.4	88.0 88.7	77.5 77.4	93.0 93.0	49.5 49.8	72.2 72.3	67.8 67.8 +0.0
2	65.7 65.7	23.6 24.1	80.7 80.8	64.6 64.8	76.5 76.3	86.8 86.6	84.2 84.7	93.9 93.7	57.5 57.4	75.4 75.4	70.9 71.0 -0.0
4	66.5 66.5	28.3 27.7	84.4 85.6	68.5 68.3	76.0 75.6	86.3 86.3	90.2 90.1	94.2 94.3	60.8 60.3	76.0 76.5	73.1 73.1 +0.0
8	67.2 67.2	35.7 36.1	89.1 88.8	74.6 74.4	76.8 76.7	88.2 87.7	92.7 92.4	94.4 94.8	63.7 63.7	80.2 80.1	76.2 76.2 +0.1
16	68.4 68.4	46.1 44.9	91.9 91.8	79.9 79.7	78.1 78.2	89.0 88.8	96.0 96.2	95.3 95.2	69.0 68.2	82.2 82.8	79.6 79.4 +0.2

Top-1 accuracy (%), 3-seed avg (seeds 1·2·3) · paper = Zanella & Ben Ayed (2024), Table 4 · averages over the 10 datasets we ran (SUN397 excluded)

RAN BY · H. B. BANKOGLU

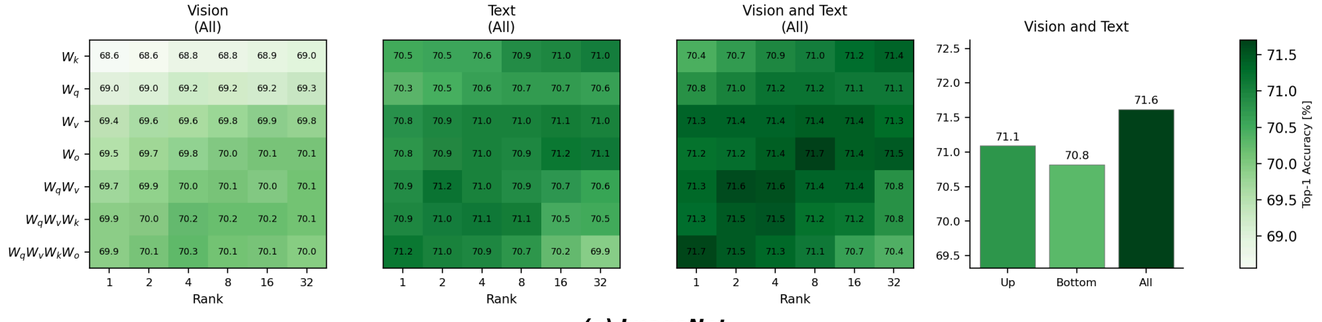
Few-shot results · ViT-L/14

ours paper Δ = ours−paper

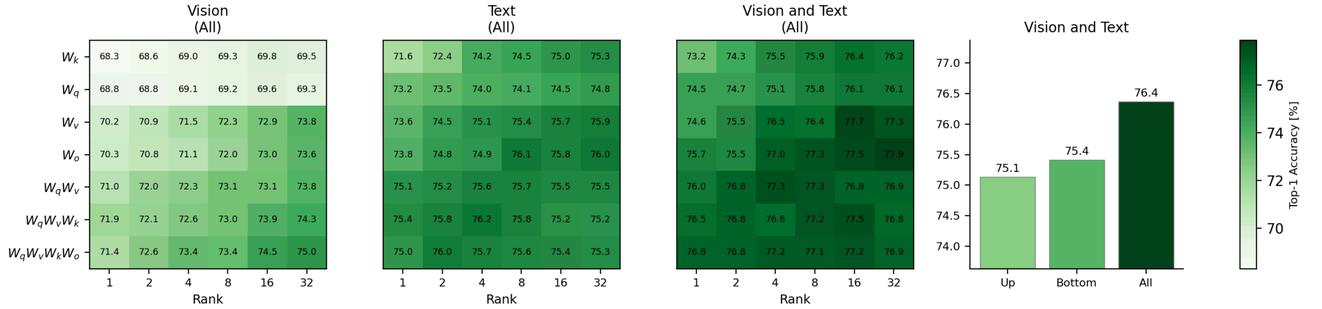
Shots	ImgNet	Airc	Euro	Cars	Food	Pets	Flwr	Calt	DTD	UCF	Avg
1	77.0 76.7	41.5 41.2	72.0 73.7	81.2 80.7	90.3 90.5	94.3 94.2	90.8 91.2	95.7 95.7	61.0 61.2	83.0 82.4	78.7 78.8 -0.1
2	77.4 77.3	43.2 42.4	85.7 85.9	82.7 82.3	89.9 89.8	93.8 93.2	95.1 94.7	96.9 96.8	67.4 67.3	84.8 84.2	81.7 81.4 +0.3
4	78.1 77.9	50.0 48.9	85.4 86.4	85.3 85.2	89.8 89.6	94.1 93.9	97.3 97.4	97.1 97.2	70.2 70.4	86.6 86.4	83.4 83.3 +0.1
8	78.7 78.5	57.9 57.5	89.5 90.0	88.8 88.7	89.5 89.7	94.1 94.2	98.3 98.0	97.0 97.0	72.7 72.2	88.7 88.3	85.5 85.4 +0.1
16	79.7 79.6	66.3 66.2	93.0 93.1	91.0 90.9	90.0 89.9	94.7 94.3	99.1 99.0	97.2 97.3	76.8 76.5	90.3 89.9	87.8 87.7 +0.2

Top-1 accuracy (%), 3-seed avg (seeds 1·2·3) · paper = Zanella & Ben Ayed (2024), Table 5 · averages over the 10 datasets we ran (SUN397 excluded)

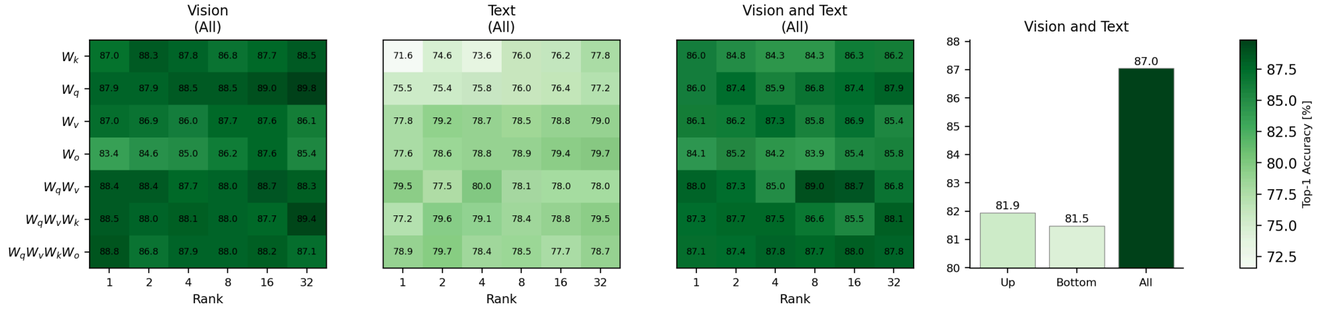
Rank × matrices × encoder — reproduced



(a) ImageNet



(b) Stanford Cars



(c) EuroSAT

4-shot · top-1 accuracy (%) · our reproduction of the paper's Figure 3 · **flat across rank ⇒ default r=2 · QKV ≈ best matrix set · both encoders & all layers win**

What made reproduction hard

DATA · #1

Dead download URLs

Caltech101, StanfordCars, SUN397 & EuroSAT original links return 404 / 403 / 500. Re-sourced via Caltech Data, Kaggle, HuggingFace, Zenodo.

DATA · #2

SUN397 in parquet

The HF mirror ships 100k+ images inside 93 parquet files — needs a custom extraction script before torchvision can read raw JPEGs.

SCALE · #3

ImageNet is 144 GB

Manual login + download of train/val/devkit (~138 GB train alone). The single heaviest item to fetch and extract.

ENV · #4

V100 CUDA mismatch

Volta (sm_70) lacked a kernel in newer PyTorch wheels → resolved by moving the runs to an **A40** with a matching CUDA build.

ENV · #5

NumPy ≥1.24 break

`float()` on a 1-D array crashed `cls_acc()`. Fixed by switching to `.item()`.

COMPUTE · #6

Run volume

150 runs *per* backbone table + ~387 ablation runs (~275 GPU-h total). Solved with a failure-tolerant runner and round-robin job splitting.

04 PART FOUR

Our extension & outlook

Anchoring the adapted model to zero-shot CLIP with a regularization term.

04 EXTENSION · THE IDEA

Keep the adapted model close to zero-shot CLIP

Problem. CLIP-LoRA trains with plain **cross-entropy**. With only a few shots, nothing stops the adapters from **drifting away** from CLIP's strong pretrained priors — a likely cause of weaker results on fine-grained sets like **Food101**.

Idea. Add a term that **anchors** the adapted output distribution to the **frozen** zero-shot CLIP — a knowledge-distillation regularizer.

$$L = \mathrm{CE}(lora\_logits, y) + \lambda \cdot T^2 \cdot \mathrm{KL}(p_{\text{zero-shot}}/T \parallel p_{\text{lora}}/T)$$

- $p_{\text{zero-shot}}$ — softmax over class similarities from **frozen** CLIP (a fixed teacher).
- p_{lora} — softmax from the **adapted** LoRA model; **T** = temperature (T^2 keeps the gradient scale).
- **λ** sets anchor strength; **λ = 0** recovers the CE-only baseline — a clean control. Chosen by ablation: **λ = 0.1, T = 8**.

KL is over class probability distributions — not weight tensors.

What we ran: ablation, then broad evaluation

STEP 1 · ABLATION

Pick λ and T

{Food101, OxfordPets, EuroSAT} × shots {1,4,16} × **λ {0.1, 0.3, 1.0}** × **T {4, 8}**, seed 1.

= 54 runs

CHOSEN

λ = 0.1 · T = 8

The only setting that helps broadly **without hurting the EuroSAT control**; higher λ over-regularizes weak-zero-shot datasets.

STEP 2 · BROAD EVAL

All 10 datasets

Re-ran the full Table-3 grid **with KL on** at λ=0.1, T=8 — 10 datasets × shots {1,2,4,8,16} × 3 seeds.

= 150 runs

COMPARE

KL vs CE-only, cell-for-cell

Same grid as Table 3, so every (dataset, shot) is a direct baseline comparison.

04 EXTENSION · RESULTS

Does the KL anchor help? CE-only vs KL on Table 3

Same grid — 10 datasets × shots {1,2,4,8,16} × 3 seeds — re-run with KL (λ=0.1, T=8). Top-1 accuracy (%), averaged per dataset.

	ImgNet	Airc	Euro	Cars	Food	Pets	Flwr	Calt	DTD	UCF	Avg
CE-only (baseline)	71.7	40.2	84.0	77.7	83.6	91.4	91.9	95.1	63.4	81.5	78.1
KL (ours)	71.8	39.6	85.2	77.8	85.6	92.1	91.9	95.4	64.2	81.6	78.5
Δ = KL − CE	**+0.1**	**-0.5**	**+1.2**	**+0.1**	**+1.9**	**+0.8**	**+0.0**	**+0.3**	**+0.9**	**+0.1**	**+0.47**

WHERE IT HELPS

Food +1.9 · Euro +1.2 · DTD +0.9 · Pets +0.8

The fine-grained / failure cases we predicted — where CE-only LoRA overfits.

HOW BROAD

35 / 50 cells improved

Only real cost is FGVC (−0.5); the rest neutral or positive.

MORE SHOTS → MORE GAIN

+0.2 → +0.6 (1→16-shot)

Benefit grows as the few-shot signal strengthens.

Verdict — yes, it improves performance: +0.47 on average, recovering accuracy precisely where plain cross-entropy lets the few-shot adapter drift from CLIP's prior, at almost no cost elsewhere.

05 CONCLUSION

Summary — what we did, and what's next

REPRODUCED · HOW

Faithful, end-to-end

Forked the authors' repo + our own runner scripts; Tables 3/4/5 (3 backbones) and Figure 3 over 10 datasets × 5 shots × 3 seeds, ~275 GPU-h. Averages within **≈0.3%** of the paper.

CHALLENGES

Mostly data & scale

Dead dataset URLs re-sourced (Kaggle/HF/Zenodo), 144 GB ImageNet, a V100→A40 CUDA fix and a NumPy break — absorbed by a failure-tolerant runner across ~700 runs.

EXTENDED

KL anchor to zero-shot CLIP

$L = CE + \lambda \cdot T^2 \cdot KL(\text{zero-shot} \parallel \text{LoRA})$; ablation picked $\lambda=0.1$, $T=8$. Stops the adapter drifting off CLIP's prior.

EXTENSION · RESULT

+0.47 avg · 35/50 cells improved

Biggest gains on Food101 +1.9, EuroSAT +1.2, DTD +0.9 — the failure cases. Confirms the hypothesis.

FUTURE WORK

- Add SUN397 → full 11-dataset set
- 3 seeds for Figure 3 (error bars)
- Per-dataset λ / T tuning
- KL on ViT-B/32 & ViT-L/14

✦ THANK YOU

Questions & discussion

PAPERS

Zanella & Ben Ayed (2024) — *Low-Rank Few-Shot Adaptation of Vision-Language Models.* arXiv:2405.18541

Radford et al. (2021) — *Learning Transferable Visual Models From Natural Language Supervision (CLIP).* arXiv:2103.00020

CODE

Base: `github.com/MaxZanella/CLIP-LoRA`

Our repository: `github.com/bnkglu/tuw-dl4nlp-2026s`

Hasan Berke Bankoglu · Zarek Asif
192.039 — DL4NLP · 2026S